

EEE373L

Embedded System Design Laboratory

Lab Report



Inspiring Excellence

Section:02

Group No:02

Experiment no:03

Name of the experiment:Input / Output of Digital Data

Prepared by:

Name: Nafim Karim Khan

ID: 21221039

Signature

All Group members:

Sl.	Name	ID	Signature
1.	Yasir Anaf Siddique	21221073	
2.	Md Talha Bin Imran	21221018	
3.	Afrida Islam	21221030	
4	Md. Hamin-Al-Araf	21221019	

Date of Submission: 22/06/2024

Objective

The objective of this experiment is to demonstrate inputting / outputting of digital data from / to the microcontroller ATmega32.

Theoretical Background

Risc (Reduced Instruction Set Computer) architecture is the foundation of the AVR microcontroller. An example of a low power CMOS controller is the ATmega32 microcontroller. AVR microcontrollers with RISC architecture are capable of processing one million instructions per second when given a crystal oscillator's cycle frequency of 1 MHz. The ATmega32 chip has 40 pins. For the four ports—PORTA, PORTB, PORTC, and PORTD—a total of 32 pins are reserved. It is possible to use these ports for both data input and output. The remaining pins are labeled as AVCC, AREF, AGND, XTAL1, XTAL2, RESET, GND, and VCC. Any of these ports must be configured appropriately in order to be used as an input or output port.

Part A: Sending Data to a Port

Circuit Diagram

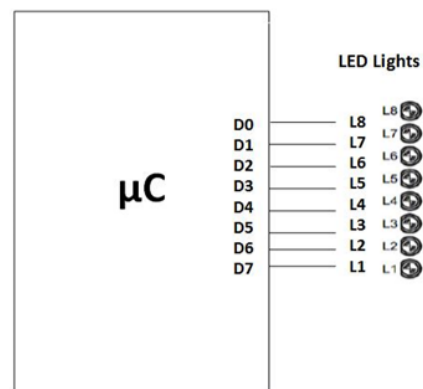


Fig.1: Circuit configuration of part A

Code

```
#include <mega32.h>
#include <delay.h>
void main(void)
{
    DDRD=0xFF;
    PORTD=0xFF;
    while (1)
```

```
{  
  delay_ms(1000);  
  PORTD=~PORTD;  
}
```

Output

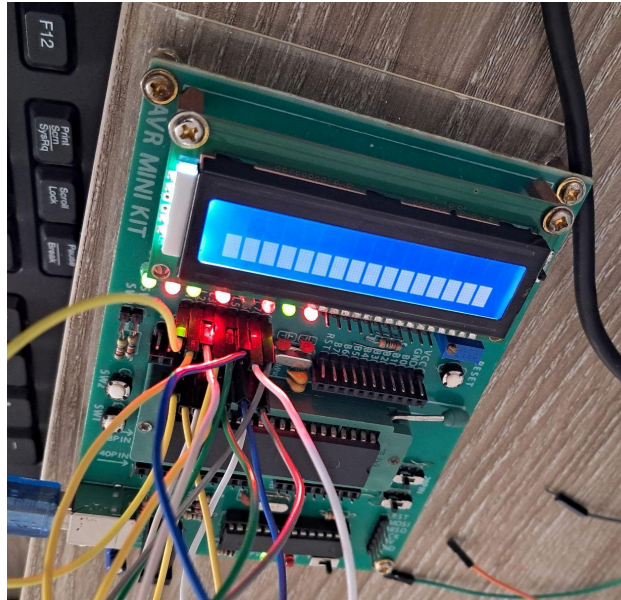


Fig 2: All LEDs ON

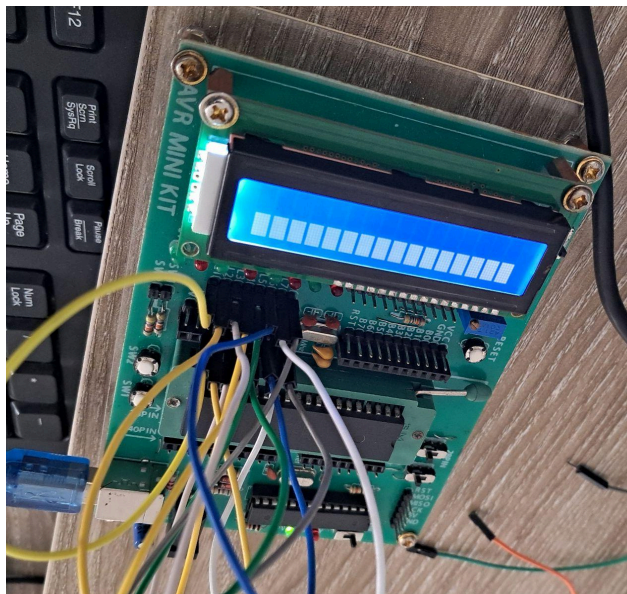


Fig 3: All LEDs OFF

Discussion

In this task, we observe how a particular set of ports can be configured to be output pins and how we can send data to these ports. Here the LEDs are connected to PORT D and are set as outputs by denoting all the PORTD to High (1). The output task shows 8 LEDs blinking with an interval of 1 sec.

Lab Report Task

Write and explain the modified program so that it will illuminate the alternate LEDs every second.

Code

```
#include <mega32.h>
#include <delay.h>

void main(void)
{
    DDRD = 0xFF;      // Set PORTD as output
    PORTD = 0x00;      // Initialize all LEDs to OFF

    while (1)
    {
        PORTD = 0xAA; // Turn on alternate LEDs (10101010)
        delay_ms(1000);

        PORTD = 0x55; // Turn on the other alternate LEDs (01010101)
        delay_ms(1000);
    }
}
```

Discussion

The modified program is designed to illuminate alternate LEDs every second on PORTD. First, DDRD = 0xFF sets all the pins of PORTD as output, meaning the program can control the LEDs connected to these pins. Initially, all LEDs are turned off by setting PORT D = 0x00. Inside the while loop, the program alternates between two patterns: PORTD = 0xAA (binary 10101010), which turns on the odd-numbered LEDs, and PORT D = 0x55 (binary 01010101), which turns on the even-numbered LEDs. A delay_ms(1000) function is used to introduce a one-second delay between switching the LED states, making the LEDs alternate every second.

Part-B: Multiple 7 segment display

Circuit Diagram

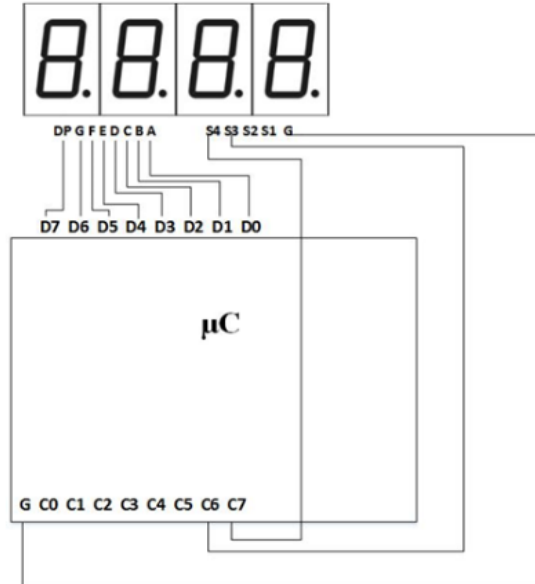


Fig 4: Circuit configuration of part B

Code

```
#include <mega32.h>
#include <delay.h>
#define data_port PORTD
#define control_port PORTC
char digit_cathode[10] = {0x3F,0x06,0x5B,0x4F,0x66,0x6D,0x7D,0x07,0x7F,0x6F};
char first,last,i=0,j;
void main(void)
{
    DDRD=0xFF;
    DDRC=0xC0;
    data_port=0;
    control_port=0;
while (1)
    {
        last=i%10; //last digit
        first=i/10; //first digit
        for(j=0;j<20;j++)
        {
            control_port=0x40; //0b01000000; S3(C6) is for the 2nd last digit
            data_port=digit_cathode[first];
            delay_ms(20);
            control_port=0x80; //S4(C7) is for the last digit
            data_port=digit_cathode[last];
            delay_ms(20);
        }
        i++;
        if (i>99) i=0;
    }
```

}
}

Output

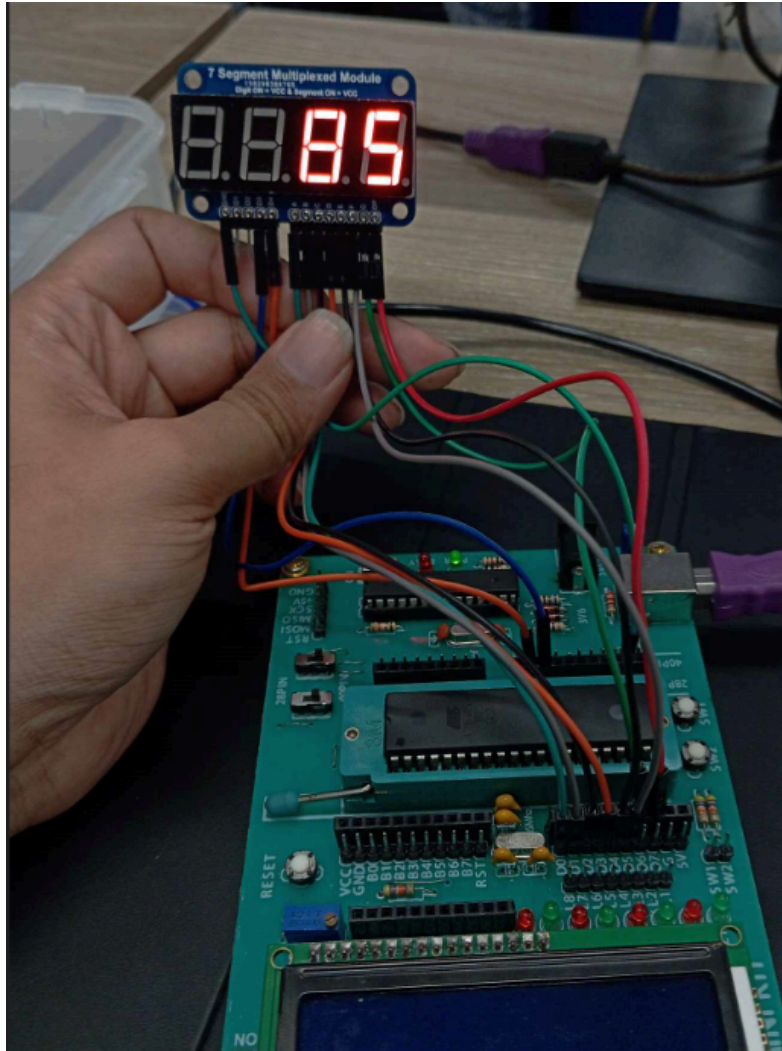


Fig 5.1: 7-Seg Display counting down from 0-99

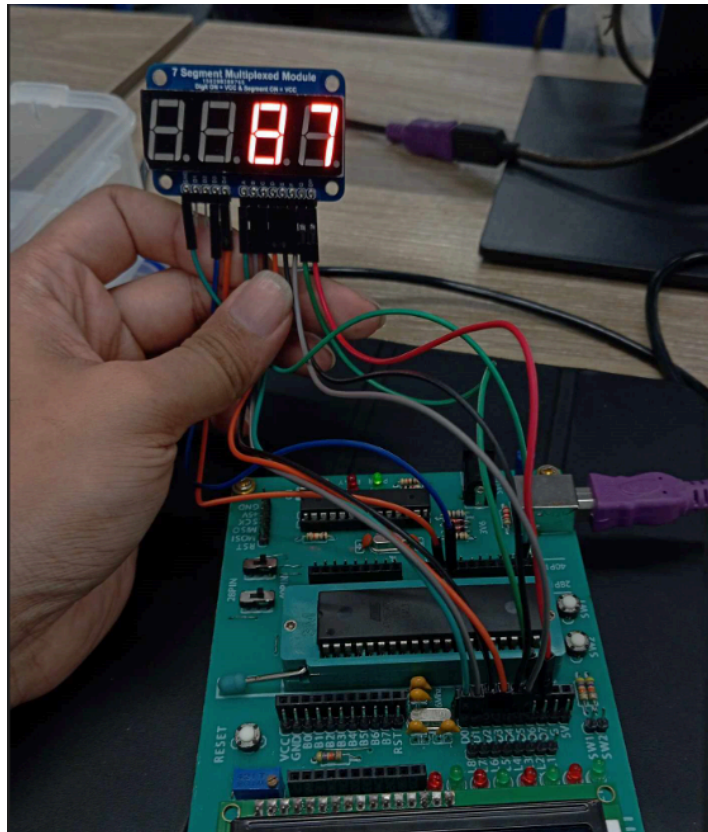


Fig 5.2: 7-Seg Display counting down from 0-99 (Continued)

Discussion

This code uses an ATmega32 microcontroller to drive a two-digit, seven-segment display. The segment patterns used to show digits 0 through 9 are stored in the array `{digit_cathode}`. The segment data is managed by `data_port` (PORTD), while `control_port` (PORTC) alternates between the two digits. Through multiplexing the display and updating each digit every 20 ms, the software shows the first and last digits of a counter (up to 99). With every loop, the `{i}` variable becomes bigger and bigger until it reaches 99, at which point it resets to 0. It then counts up to 99 and keeps the numbers on the screen.

Report Task

Write and explain a modified program so that it will display 000-999.

Code

```
#include <mega32.h>
#include <delay.h>
#define data_port PORTD
#define control_port PORTC
char digit_cathode[10] = {0x3F,0x06,0x5B,0x4F,0x66,0x6D,0x7D,0x07,0x7F,0x6F}; // Segment
codes for digits 0-9
char first, second, last, i = 0, j;
```

```

void main(void)
{
    DDRD = 0xFF;      // Set PORTD as output
    DDRC = 0xE0;      // Set PORTC (C5, C6, C7) as output

    data_port = 0;
    control_port = 0;

    while (1)
    {
        last = i % 10;      // Get the last digit (ones)
        second = (i / 10) % 10; // Get the second digit (tens)
        first = i / 100;    // Get the first digit (hundreds)

        for (j = 0; j < 20; j++)
        {
            // Control for the first digit (hundreds place)
            control_port = 0x20; // 0b00100000; S2 (C5) controls hundreds
            data_port = digit_cathode[first];
            delay_ms(10);

            // Control for the second digit (tens place)
            control_port = 0x40; // 0b01000000; S3 (C6) controls tens
            data_port = digit_cathode[second];
            delay_ms(10);

            // Control for the last digit (ones place)
            control_port = 0x80; // 0b10000000; S4 (C7) controls ones
            data_port = digit_cathode[last];
            delay_ms(10);
        }

        i++; // Increment the counter
        if (i > 999) i = 0; // Reset after reaching 999
    }
}

```

Discussion

This modified program controls a three-digit 7-segment display, allowing it to count from 000 to 999. The digits are extracted from the counter `i`, with `first` representing the hundreds, `second` the tens, and `last` the ones. The display is multiplexed by activating each digit in quick succession using `control\_port` (C5, C6, C7). Each digit is displayed for 10 ms, making all digits appear lit simultaneously. The counter increments from 0 to 999, resetting after reaching 999, displaying numbers continuously.

Part C: Input of digital data

Circuit Diagram

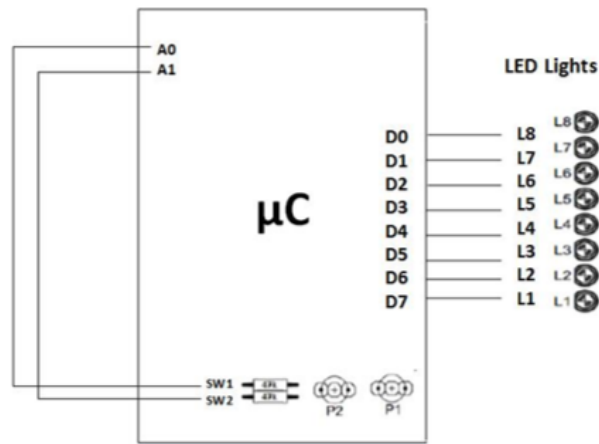


Fig.6: Circuit configuration of part C

Code

```
#include <mega32.h>
#include <delay.h>
void main(void)
{
    char move=0;
    DDRD=0xFF;
    PORTD=0x80;
    DDRA=0x00;
    PORTA=0x03; //to turn on the pull up resistor
    while (1)
    {
        if (PINA.0==0) move=1;
        if (PINA.1==0) move=2;

        if (move==1) //if (PINA.0==0)
        {
            PORTD=PORTD>>1;
            if (PORTD==0)
                PORTD=0x80;
        }
        if (move==2)
        {
            PORTD=PORTD<<1;
            if (PORTD==0)
                PORTD=0x01;
        }
        delay_ms(200);
    }
}
```

Output

- When Switch 1 is pressed, LEDs shift right

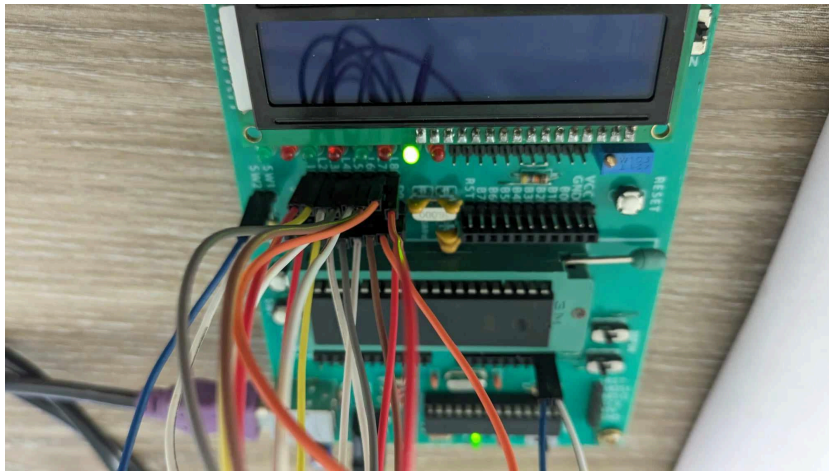


Fig 6.1: LED Shifting right when SW1 is Pressed

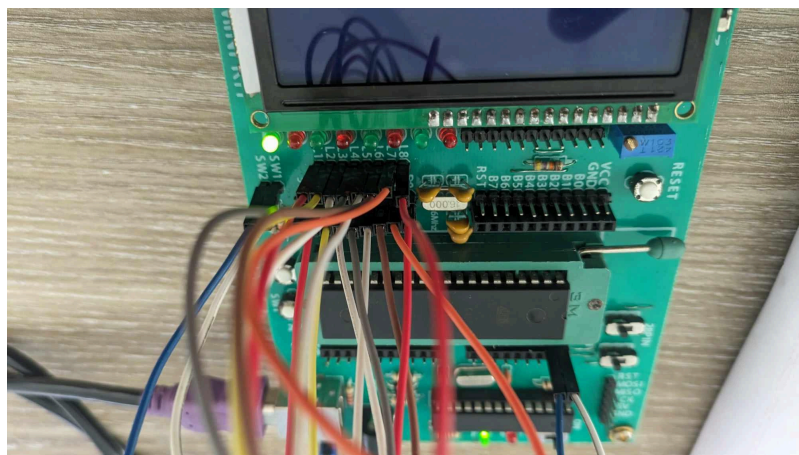


Fig 6.2: LED Shifting right when SW1 is Pressed (Continued)

- When Switch 2 is pressed, LEDs shift left

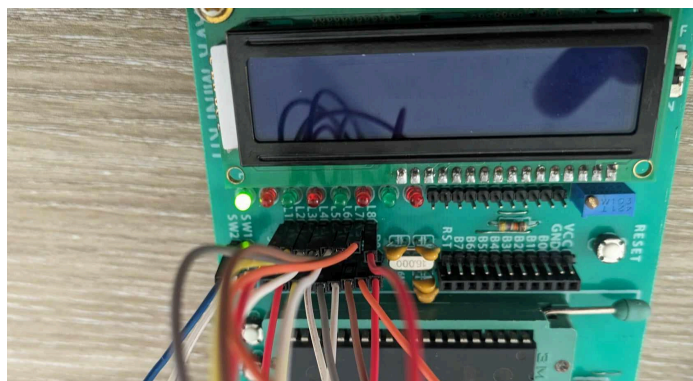


Fig 6.3: LED shifting leftwards when SW2 Pressed

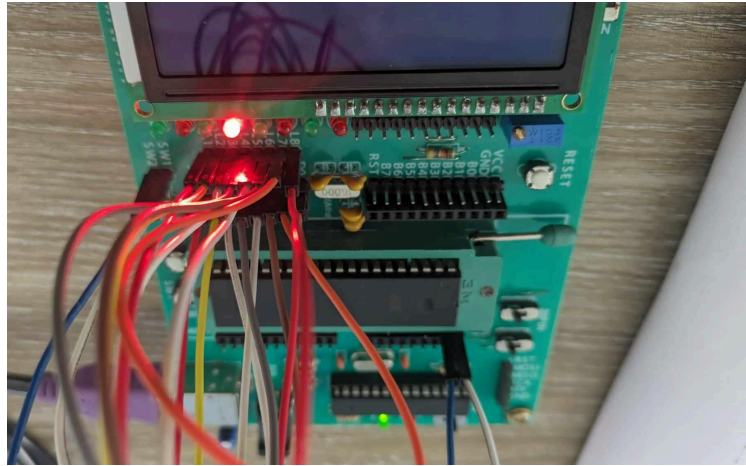


Fig 6.4: LED shifting leftwards when SW2 pressed (Continued)

Discussion

This program controls the shifting of an LED on 'PORTD' based on the input from two push buttons connected to 'PINA.0' and 'PINA.1'. When button 'PINA.0' is pressed, the LED shifts right, and when button 'PINA.1' is pressed, the LED shifts left. The pull-up resistors on 'PINA' are enabled, ensuring stable button readings. The 'PORTD' value shifts either left or right by one bit, wrapping around once it reaches the end. The LED shift operation occurs every 200 milliseconds, making the movement visible.

Report task

Write and explain a modified program so that it will reverse the order of LED shift operation.

Code

```
#include <mega32.h>
#include <delay.h>

void main(void)
{
    char move = 0;
    DDRD = 0xFF;           // Set PORTD as output
    PORTD = 0x01;          // Start with the first LED on (right-most)
    DDRA = 0x00;           // Set PORTA as input
    PORTA = 0x03;          // Enable pull-up resistors on PINA.0 and PINA.1

    while (1)
    {
        if (PINA.0 == 0) move = 1; // Button PINA.0 pressed
        if (PINA.1 == 0) move = 2; // Button PINA.1 pressed

        if (move == 1) // Move left when PINA.0 is pressed
        {
```

```

        PORTD = PORTD << 1;      // Shift left
        if (PORTD == 0)          // Reset if no LEDs are on
            PORTD = 0x01;
    }
    if (move == 2) // Move right when PINA.1 is pressed
    {
        PORTD = PORTD >> 1;      // Shift right
        if (PORTD == 0)          // Reset if no LEDs are on
            PORTD = 0x80;
    }
    delay_ms(200); // Delay between shifts
}
}

```

Discussion

This modified program reverses the direction of the LED shift operations. Initially, PORTD is set to 0x01, turning on the right-most LED. When button PINA.0 is pressed, the LED shifts left (PORTD << 1), and when button PINA.1 is pressed, the LED shifts right (PORTD >> 1). If the LED reaches either end (no LEDs are on), it wraps around to the opposite end: PORTD = 0x01 (right-most) when shifting left, and PORTD = 0x80 (left-most) when shifting right. The 200 ms delay controls the speed of the shifts.

Bonus Class Task

- Display HIGH on 7 Segment display

The Code

```

#include <mega32.h>
#include <delay.h>

char digit_cathode[10] = {0x3F,0x06,0x5B,0x4F,0x66,0x6D,0x7D,0x07,0x7F,0x6F};
void main(void)
{
    DDRC=0xFF; //used for output data
    DDRD=0x0F; //last 4 LSB used for switching displays, also acting as outputs

    while (1){
        PORTC=0x76; //Hex code for displaying 'H' on CC 7 segment display
        PORTD=0b00000001;
        delay_ms(10);
        PORTC=0x7D; //G
        PORTD=0b00000010;
        delay_ms(10);
        PORTC=0x06; //I
        PORTD=0b00000100;
        delay_ms(10);

        PORTC=0x76; //H
        PORTD=0b00001000;
    }
}

```

```
delay_ms(10);  
}  
}
```

Output

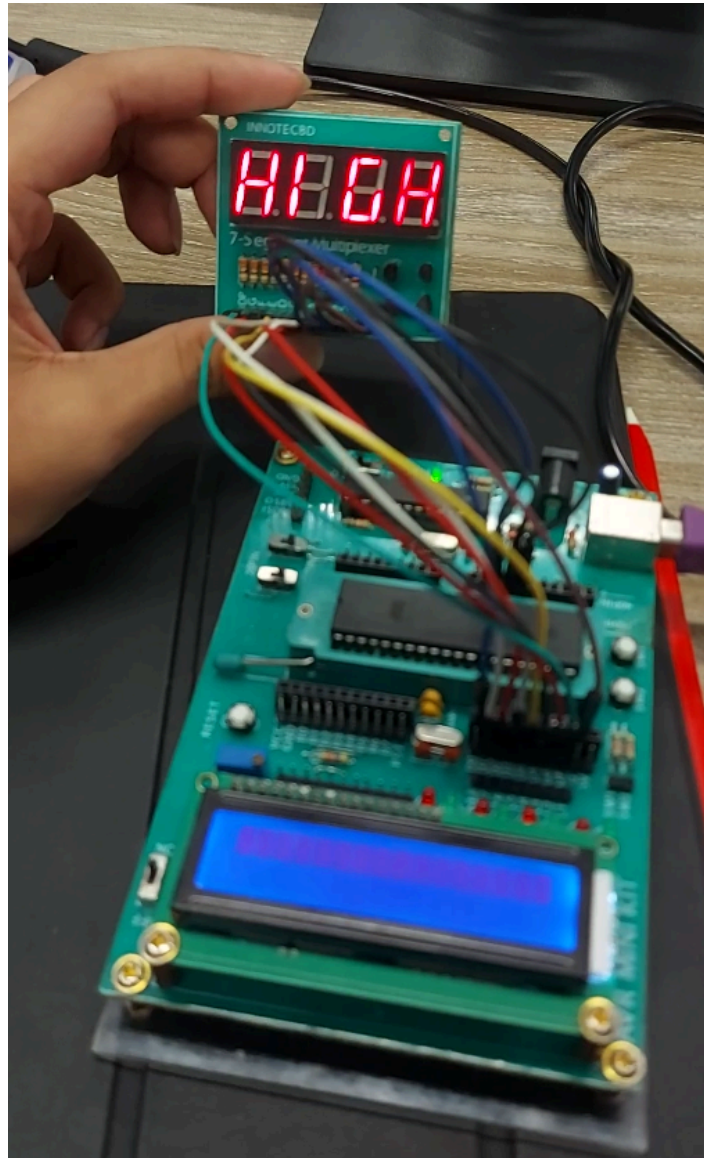


Fig 7: Displaying HIGH on 7-Segment Displays

- To Blink 'HI' on 7-Segment display

The Code

```
#include <mega32.h>
#include <delay.h>
#define data_port PORTD
#define control_port PORTC
char digit_cathode[10] = {0x3F,0x06,0x5B,0x4F,0x66,0x6D,0x7D,0x07,0x7F,0x6F};
char first,last,i=0,j;
void main(void)
{
    DDRD=0xFF;
    DDRC=0xC0;
    data_port=0;
    control_port=0;
while (1)
    {
        for(i=0;i<20;i++){
            control_port=0b10000000; //0b01000000; S3(C6) is for the 2nd last digit
            data_port=0x76;
            delay_ms(10);
            control_port=0b01000000;
            data_port=0x06;
            delay_ms(10);
            control_port=0b11000000;
            data_port=0x00;
        }

        delay_ms(1000);

    }
}
```


The Output:

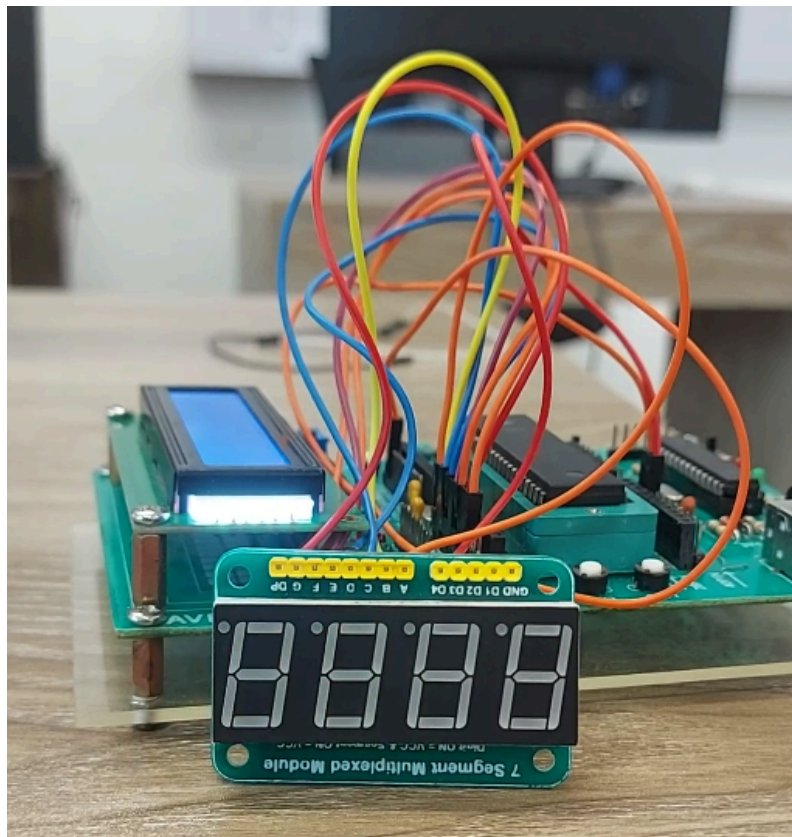


Fig 8.1: Blinking HI on 7 Segment display (7-Segment displays OFF)

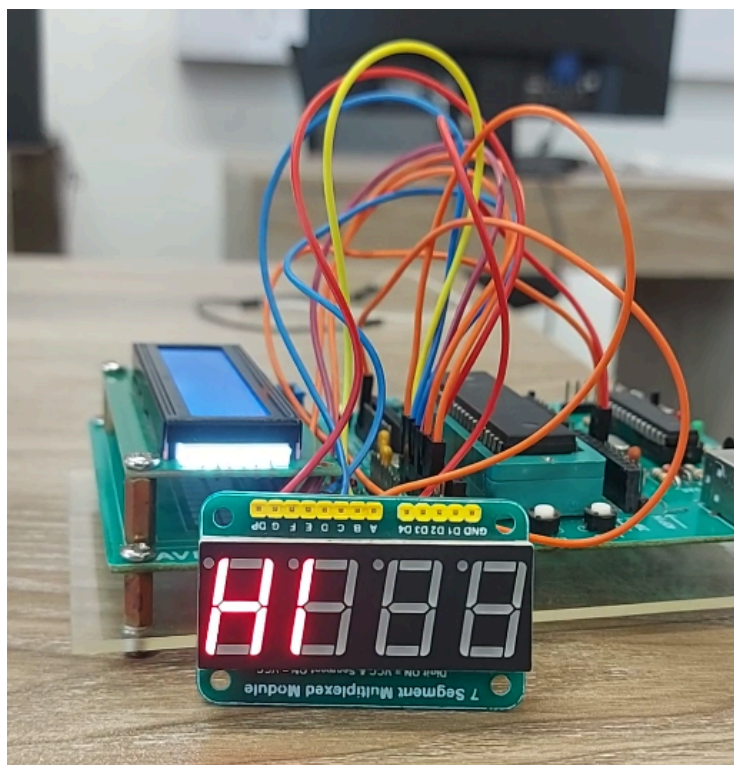


Fig 8.2: Blinking HI on 7 Segment display (7-Segment displays ON)